Machine Intelligence I
Lecture Notes

Contents

Preface **2**

Notation **2**

1 Foundations: From Learning Problems to Multilayer Perceptrons **3**
1.1 Learning as model selection . . . 3
1.2 Connectionist neurons . . . 4
1.3 Why multilayer perceptrons are needed . . . 4
1.4 Backpropagation . . . 5

2 Optimization, Classification, and Regularization **5**
2.1 Empirical risk minimization . . . 5
2.2 Gradient-based optimization . . . 6
2.3 Probabilistic classification . . . 6
2.4 Generalization, bias, and variance . . . 6
2.5 Regularization and validation . . . 7

3 Deep Learning and Convolutional Representations **7**
3.1 Why depth helps . . . 7
3.2 Vanishing gradients and practical remedies . . . 8
3.3 Hierarchical features . . . 8
3.4 Autoencoders and pre-training . . . 9
3.5 Convolutional layers . . . 9
3.6 Regularization in deep networks . . . 9

4 Sequence Models and Local Basis Methods **10**
4.1 Recurrent neural networks . . . 10
4.2 Backpropagation through time . . . 10
4.3 Gating and stable memory . . . 10
4.4 Radial basis function networks . . . 10
4.5 What these models teach . . . 11

5 Statistical Learning Theory and Support Vector Machines **11**
5.1 True risk and empirical risk . . . 11
5.2 Capacity and VC dimension . . . 12
5.3 Structural risk minimization . . . 12

5.4 Maximum-margin classification . . . 12
5.5 Soft margins and kernels . . . 13
5.6 Main lesson . . . 13

6 Bayesian Inference, Graphical Models, and Learning under Uncertainty 13
6.1 Bayes' rule . . . 13
6.2 Conditional independence . . . 14
6.3 Bayesian networks . . . 14
6.4 Bayesian prediction and model selection . . . 14
6.5 MAP estimation and regularization . . . 15
6.6 What Bayesian methods add . . . 15

7 Reinforcement Learning 15
7.1 Markov decision processes . . . 15
7.2 Value functions . . . 16
7.3 Temporal-difference learning . . . 16
7.4 Control: SARSA and Q-learning . . . 16
7.5 Exploration and exploitation . . . 16
7.6 Course synthesis . . . 17

Mathematical Synthesis 17

References 19

Preface

These notes rewrite *Machine Intelligence I* as a compact mathematical narrative rather than a slide-by-slide transcript. The goal is to keep only the conceptual spine of the course: how learning problems are posed, how models are trained, why generalization is hard, how uncertainty is represented, and how sequential decision-making is formalized.

The emphasis is deliberately uneven in a useful way. Equations are made explicit whenever they carry the main idea. Prose is kept short. Intuition is stated immediately after the mathematics so that the symbols do not float free of meaning. The resulting document is meant to function as a fast, rigorous study guide for review, exam preparation, and later reuse.

Berlin, April 2026

Notation

Learning and optimization

Symbol	Meaning
$\mathbf{x}$	input vector
y	target or output
$\mathbf{w}$	parameter vector or weight vector
$\ell(\mathbf{w}; \mathbf{x}, y)$	loss on one sample
$R(\mathbf{w})$	true risk
$R_{\text{emp}}(\mathbf{w})$	empirical risk

Neural networks

Symbol	Meaning
a_j	pre-activation
h_j	hidden activation
$\sigma(\cdot)$	elementwise nonlinearity
$\mathbf{W}$	weight matrix
δ_j	backpropagated error signal

Probability and reinforcement learning

Symbol	Meaning
$p(\cdot)$	probability density or mass function
$\mathbb{E}[\cdot]$	expectation
s_t, a_t, r_t	state, action, reward at time t
$V^\pi(s)$	value of state s under policy π
$Q^\pi(s,a)$	action value under policy π
γ	discount factor

1 Foundations: From Learning Problems to Multilayer Perceptrons

Machine intelligence starts from a simple observation: the system designer usually does not know the correct input–output rule in advance. The task is therefore to specify a model family, define a criterion of success, and learn parameters from data rather than write the full rule by hand.

1.1 Learning as model selection

At a high level, a learning problem has three ingredients:

1. a hypothesis class,
2. a loss function,
3. and an optimization procedure.

Learning pipeline

$$\text{data} \longrightarrow \text{model class} \longrightarrow \text{loss} \longrightarrow \text{optimization}$$

Learning is not "fit a curve" in the abstract. It is the joint choice of representation, objective, and search procedure.

This viewpoint already separates the main paradigms of the course:

- **supervised learning**: learn from labeled pairs $(\mathbf{x}^{(\alpha)}, y^{(\alpha)})$,
- **unsupervised learning**: discover structure in $\mathbf{x}^{(\alpha)}$ alone,
- **reinforcement learning**: learn from trajectories with delayed reward.

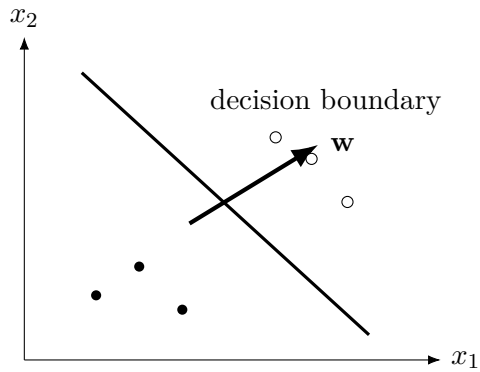

Figure 1: A single neuron implements a linear decision boundary. The weight vector is normal to the separating hyperplane.

Core intuition. The paradigms differ mainly by what feedback is available. Labels give direct error signals, unsupervised objectives invent structure from the inputs themselves, and reinforcement learning must assign credit across time.

1.2 Connectionist neurons

The elementary computational unit of early machine learning is the connectionist neuron,

$$y_i = f\left(\sum_j w_{ij} x_j - \theta_i\right).$$

Here $\mathbf{x}$ is the input, $\mathbf{w}_i$ is a learned weight vector, θ_i is a threshold or bias term, and f is a transfer function such as a threshold, sigmoid, tanh, or ReLU.

Two interpretations matter:

1. **feature detector**: $\mathbf{w}_i$ selects a preferred direction in input space,
2. **decision unit**: the sign of $\mathbf{w}_i^\top \mathbf{x} - \theta_i$ tells us on which side of a hyperplane the input lies.

The geometric takeaway is immediate: changing θ_i shifts the hyperplane, while changing $\mathbf{w}_i$ rotates it. A single neuron can therefore separate only linearly separable classes.

1.3 Why multilayer perceptrons are needed

One layer is not enough for problems such as XOR. A multilayer perceptron (MLP) composes affine maps and nonlinearities,

$$\mathbf{h}^{(\ell)} = \sigma\big(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\big),$$

so each layer transforms the representation before the next layer acts on it.

Representation view. Depth is useful because later layers are not solving the original problem directly. They are solving a problem in a learned feature space created by earlier layers.

Feedforward networks propagate information from input to output. Recurrent networks reuse internal state across time and are treated later in the course.

1.4 Backpropagation

The practical difficulty with MLPs is not representation but optimization. If a network produces loss L, training needs the derivative of L with respect to every weight. Backpropagation solves this with repeated use of the chain rule.

Backpropagation recursion

$$\delta_j^{(\ell)} = \sigma'(a_j^{(\ell)}) \sum_k w_{kj}^{(\ell+1)} \delta_k^{(\ell+1)}, \qquad \frac{\partial L}{\partial w_{ji}^{(\ell)}} = \delta_j^{(\ell)} h_i^{(\ell-1)}.$$

The error signal at one layer is a weighted sum of downstream error signals, rescaled by the local slope of the nonlinearity.

The standard training loop is therefore:

1. compute activations in a forward pass,
2. evaluate the loss,
3. propagate error signals backward,
4. update weights using a gradient step.

The most important intuition is local: every weight update asks, "if this parameter changed slightly, how would the final loss change?" Backpropagation makes that question tractable in large layered systems.

2 Optimization, Classification, and Regularization

Once a model family has been fixed, the next question is how to train it reliably and how to keep it from merely memorizing the training set. This chapter connects empirical risk minimization, gradient-based optimization, probabilistic classification, and regularization.

2.1 Empirical risk minimization

For data $(\mathbf{x}_i, y_i)_{i=1}^N$ and parameters $\mathbf{w}$, the training objective is usually written as

$$R_{\text{emp}}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \ell(\mathbf{w}; \mathbf{x}_i, y_i).$$

If regularization is added, one instead minimizes

$$J(\mathbf{w}) = R_{\text{emp}}(\mathbf{w}) + \lambda \Omega(\mathbf{w}).$$

Why this matters. The empirical risk is only a sample average. Minimizing it is sensible only if low training loss is informative about low future loss. The rest of the chapter explains why that can fail and what one does about it.

2.2 Gradient-based optimization

Gradient descent updates parameters by

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla J(\mathbf{w}_t).$$

The main regimes differ only in how the gradient is estimated:

- **batch gradient descent**: use all samples for each update,
- **stochastic gradient descent**: use one sample at a time,
- **mini-batch methods**: use a small subset of samples per update.

Mini-batches are the practical default because they trade variance against compute well. Momentum, adaptive step sizes, and methods such as Adam improve this baseline by smoothing noisy gradients and scaling updates per parameter [GBC16; Bis06].

2.3 Probabilistic classification

In multi-class classification, a network often outputs logits $z_k(\mathbf{x})$ that are converted to probabilities by the softmax map

$$p(C_k \mid \mathbf{x}, \mathbf{w}) = \frac{\exp z_k(\mathbf{x})}{\sum_j \exp z_j(\mathbf{x})}.$$

If the target is one-hot encoded as $y_k \in \{0, 1\}$ with $\sum_k y_k = 1$, the cross-entropy loss becomes

$$L(\mathbf{w}) = -\sum_k y_k \log p(C_k \mid \mathbf{x}, \mathbf{w}).$$

Softmax plus cross-entropy

$$L(\mathbf{w}) = -\sum_k y_k \log p(C_k \mid \mathbf{x}, \mathbf{w})$$

The loss is small only when the model assigns high probability to the correct class. Classification is therefore turned into likelihood maximization over a categorical model.

The important conceptual shift is that the network no longer outputs only a label. It outputs a distribution over labels. That makes uncertainty explicit and allows training by a smooth differentiable objective.

2.4 Generalization, bias, and variance

A model may achieve low training error and still fail on new data. The missing quantity is the *generalization error*, the expected loss on unseen samples from the same population.

The standard vocabulary is:

- **underfitting**: the model class is too rigid,
- **overfitting**: the model fits noise in the sample,
- **bias**: systematic error from limited flexibility,
- **variance**: sensitivity to the specific training sample.

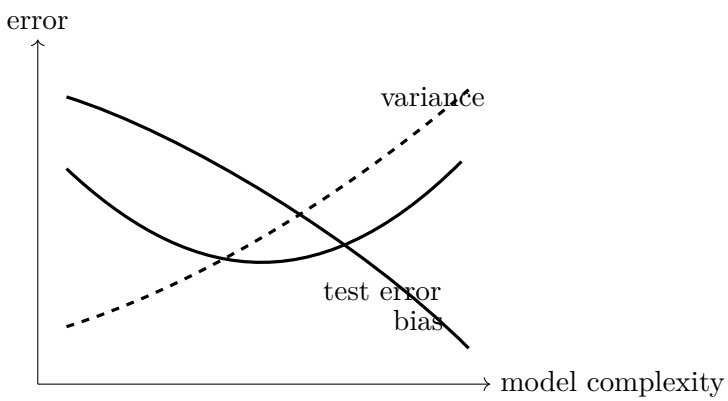

Figure 2: Increasing model complexity reduces bias but increases variance. The test error is minimized between underfitting and overfitting.

2.5 Regularization and validation

Regularization adds inductive bias that prefers simpler explanations. Common tools are:

- **weight decay**: penalize large weights with $\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2$,
- **early stopping**: stop when validation error ceases to improve,
- **cross-validation**: estimate out-of-sample performance by repeated train/validation splits,
- **train/validation/test separation**: use different subsets for fitting, model selection, and final evaluation.

Geometric intuition of weight decay. Without regularization, many parameter vectors may fit the same sample. Weight decay prefers solutions that stay near the origin, which often correspond to smoother and less brittle decision rules.

The chapter-level lesson is simple: training is not only about descending a loss surface. It is about finding parameters that both fit the observed sample and remain credible beyond it.

3 Deep Learning and Convolutional Representations

Depth became central in machine learning once practitioners learned how to train large models and once data and compute became abundant enough for those models to be useful. The point of depth is not mystery. It is representation: deeper models can construct progressively more abstract features.

3.1 Why depth helps

In a deep feedforward network, repeated composition produces

$$\mathbf{h}^{(L)} = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}).$$

Each layer reshapes the representation seen by the next layer. This can be far more parameter-efficient than forcing one shallow layer to realize the same input–output map directly [GBC16; HTF09].

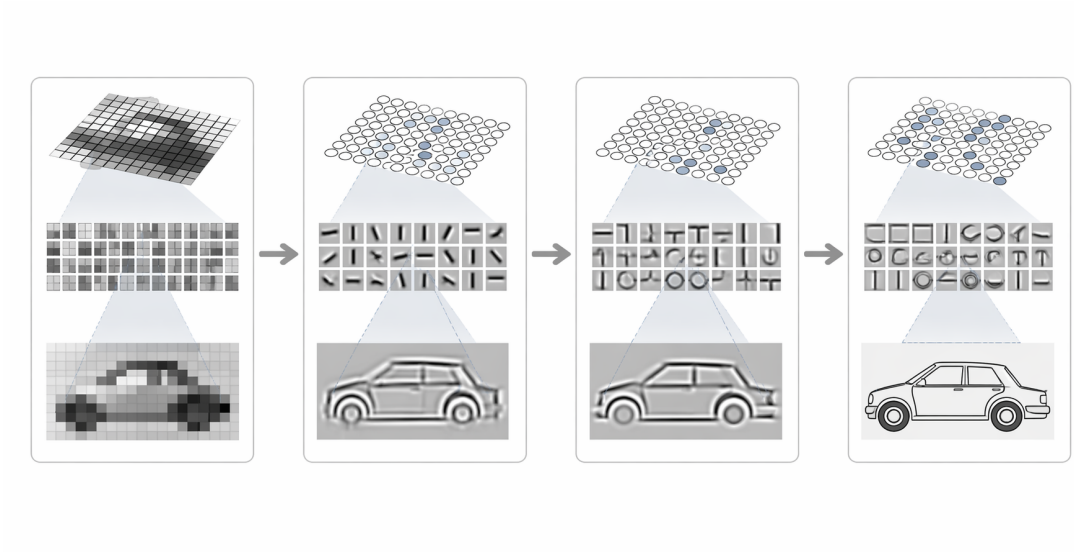

Figure 3: Representative deep-learning intuition: successive layers transform raw input into progressively more abstract features. When the Codex-generated figure is present, it replaces the schematic fallback.

Trainability is separate from expressivity. Universal approximation tells us that shallow networks can represent many functions. It does not say that those functions are easy to learn from finite data with finite computation.

3.2 Vanishing gradients and practical remedies

Deep networks were historically difficult to train because repeated chain-rule multiplication can shrink or amplify gradients exponentially. If each local derivative has magnitude below one, gradients decay with depth; if it exceeds one, they can explode.

Several practical changes broke this bottleneck:

- non-saturating nonlinearities such as ReLU,
- better initialization,
- normalization and adaptive optimization,
- pre-training in earlier deep-learning practice,
- larger datasets and hardware acceleration.

3.3 Hierarchical features

The intuitive picture is a hierarchy:

- early layers capture local regularities,
- intermediate layers combine them into motifs or parts,
- late layers encode task-relevant abstractions.

This is why depth often helps even when the final task is "just" classification: the network is learning a useful internal language for the data.

3.4 Autoencoders and pre-training

An autoencoder learns an encoder g_ϕ and decoder h_ψ so that

$$\mathbf{x} \approx h_\psi\big(g_\phi(\mathbf{x})\big).$$

If the latent representation is lower-dimensional or otherwise constrained, the model must preserve the information that matters most for reconstruction. Earlier deep-learning practice used such objectives for unsupervised pre-training; the broader lesson survives even today: representation learning can be useful before the final supervised task is optimized.

3.5 Convolutional layers

Images and other grid-structured signals have strong locality and approximate translation symmetry. Convolutional layers exploit that structure by sharing one local filter across positions. In two dimensions,

$$(\mathbf{K} * \mathbf{X})_{ij} = \sum_{u,v} K_{uv} X_{i-u,j-v}.$$

Why convolution works

$$(\mathbf{K} * \mathbf{X})_{ij} = \sum_{u,v} K_{uv} X_{i-u,j-v}$$

The same filter is reused at every position. This drastically reduces parameter count and biases the network toward local feature detection.

Pooling layers then summarize nearby activations, which improves robustness to small shifts and deformations. The key inductive bias is not that CNNs are universally better, but that they are matched to data with local correlation structure.

3.6 Regularization in deep networks

Deep models remain vulnerable to overfitting, so the usual tools reappear in more specialized form:

- **data augmentation**: enlarge the effective dataset by label-preserving transformations,
- **dropout**: prevent units from depending too strongly on specific partners,
- **weight decay and early stopping**: still essential.

The deep-learning message of the course is therefore not "depth solves everything." It is: depth is powerful when architecture, optimization, and data geometry are aligned.

4 Sequence Models and Local Basis Methods

Two important departures from standard feedforward learning appear in this part of the course. Recurrent neural networks handle temporal dependence by carrying forward a latent state. Radial basis function networks handle nonlinear regression and classification by stitching together local basis functions.

4.1 Recurrent neural networks

A basic recurrent network updates its hidden state according to

$$\mathbf{h}_t = \sigma(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}), \qquad \mathbf{y}_t = g(\mathbf{W}_o \mathbf{h}_t + \mathbf{c}).$$

The hidden state is a learned summary of the past. Unlike a fixed sliding window, it does not require the horizon of relevant history to be chosen in advance.

State as compressed memory. The network does not store the whole past explicitly. It learns which aspects of the past are worth preserving in $\mathbf{h}_t$ for future predictions.

4.2 Backpropagation through time

Training an RNN unfolds the recurrence across time and applies backpropagation to the resulting deep computational graph. If the sequence loss is

$$L = \sum_{t=1}^{T} \ell_t,$$

then gradients at time t depend on all later losses whose computation involved $\mathbf{h}_t$.

This is conceptually straightforward and numerically delicate. Repeated multiplication by the recurrent Jacobian can cause vanishing or exploding gradients, which makes long-range dependencies difficult to learn.

4.3 Gating and stable memory

Architectures such as LSTMs address this by introducing controlled memory paths. In a simplified form,

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

The forget, input, and output gates regulate what is retained, written, and exposed. The mathematical role of gating is to create a route through time along which information can persist without being repeatedly distorted.

4.4 Radial basis function networks

An RBF network uses localized hidden units

$$\phi_i(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{t}_i\|^2}{2\sigma_i^2}\right), \qquad y(\mathbf{x}) = \sum_{i=1}^{M} w_i \phi_i(\mathbf{x}).$$

Each hidden unit is centered at a prototype $\mathbf{t}_i$ and responds only near that location.

Local approximation

$$y(\mathbf{x}) = \sum_{i=1}^{M} w_i \phi_i(\mathbf{x})$$

RBF networks do not build a deep hierarchy. They tile the input space with local basis functions and then perform linear readout in that nonlinear feature space.

The standard training strategy is two-stage:

1. choose centers $\mathbf{t}_i$ by unsupervised clustering or prototypes,
2. solve the output weights by linear regression once the basis functions are fixed.

With design matrix $\Phi_{\alpha i} = \phi_i(\mathbf{x}^{(\alpha)})$, the least-squares solution satisfies

$$\Phi^\top \Phi\, \mathbf{w} = \Phi^\top \mathbf{y}.$$

4.5 What these models teach

RNNs and RBF networks solve different problems, but they teach the same structural lesson: architecture is a form of prior knowledge. Recurrent connections encode temporal dependence. Radial basis functions encode locality. In both cases, performance depends on how well that prior matches the data geometry.

5 Statistical Learning Theory and Support Vector Machines

Optimization alone does not explain why learning works. Statistical learning theory asks when empirical risk minimization is trustworthy and how model capacity should be controlled. Support vector machines are the clearest algorithmic realization of these ideas.

5.1 True risk and empirical risk

For predictor $f_{\mathbf{w}}$ and data distribution $p(\mathbf{x}, y)$, the true risk is

$$R(\mathbf{w}) = \int \ell(\mathbf{w}; \mathbf{x}, y)\, p(\mathbf{x}, y)\ \mathrm{d}\mathbf{x}\ \mathrm{d}y.$$

Because $p(\mathbf{x}, y)$ is unknown, learning replaces this expectation by the sample average

$$R_{\mathrm{emp}}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \ell(\mathbf{w}; \mathbf{x}_i, y_i).$$

The central question. When does minimizing R_{emp} also produce low R? The answer depends on how expressive the hypothesis class is relative to the amount of data available.

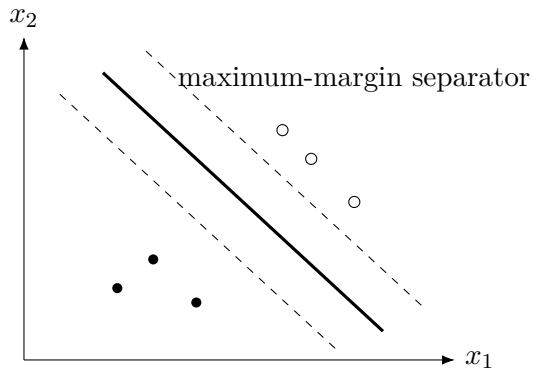

Figure 4: The support vector machine selects the separating hyperplane with the widest margin.

5.2 Capacity and VC dimension

Capacity measures how rich a model class is. A class with very high capacity can fit many labelings of the same points, which makes low training error less informative. For linear classifiers in general position, Cover's counting formula gives

$$C(p, N) = 2 \sum_{k=0}^{N-1} \binom{p-1}{k},$$

the number of dichotomies of p points in $\mathbb{R}^N$ that can be realized by a hyperplane.

The VC dimension d_{VC} is the largest number of points that can be shattered. Finite VC dimension provides one route to uniform generalization guarantees [Vap98; HTF09].

5.3 Structural risk minimization

The key idea is not to minimize training error in the largest possible model class. Instead, one balances fit against capacity across nested classes. This is structural risk minimization: the algorithm should prefer a predictor that fits well *and* lives in a class that is not unnecessarily complex.

5.4 Maximum-margin classification

For linearly separable data, a support vector machine finds the separating hyperplane with maximum margin. If the classifier is

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b,$$

the hard-margin SVM solves

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \qquad \text{for all } i.$$

SVM objective

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$$

Minimizing $\|\mathbf{w}\|$ is equivalent to maximizing the geometric margin. The support vectors are precisely the points that make this constraint active.

5.5 Soft margins and kernels

Real data are rarely perfectly separable, so slack variables ξ_i are introduced:

$$\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i, \qquad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0.$$

The parameter C trades margin size against training violations.

Kernel methods then replace inner products by $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$, allowing a linear separator in feature space to become a nonlinear separator in input space.

5.6 Main lesson

Support vector machines matter not only as an algorithm but as a clean conceptual package:

- generalization depends on capacity, not training error alone,
- margins provide a practical notion of controlled complexity,
- optimization can encode learning-theoretic principles directly.

6 Bayesian Inference, Graphical Models, and Learning under Uncertainty

Deterministic predictions are often not enough. In many problems the main question is not only "what is the most likely answer?" but also "how certain is the model, and how should that certainty change when new evidence arrives?" Bayesian inference provides the formal language for that update.

6.1 Bayes' rule

For hypothesis h and data D,

$$p(h \mid D) = \frac{p(D \mid h)\, p(h)}{p(D)}.$$

The prior $p(h)$ encodes knowledge before the observation. The likelihood $p(D \mid h)$ scores how compatible the data are with that hypothesis. The posterior combines both.

Posterior update

$$p(h \mid D) \propto p(D \mid h)\, p(h)$$

Bayesian inference is a competition between prior plausibility and data fit. The posterior reweights beliefs rather than replacing them from scratch.

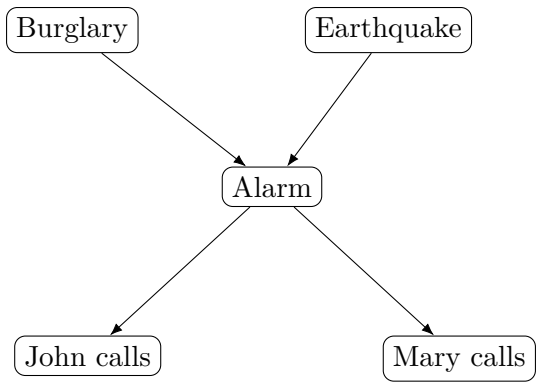

Figure 5: A canonical Bayesian-network example. The graph represents factorization structure, not merely visual correlation.

6.2 Conditional independence

Probabilistic models become useful only when they factorize. The simplest nontrivial notion is conditional independence:

$$X \perp Y \mid Z \quad \iff \quad p(x, y \mid z) = p(x \mid z)\, p(y \mid z).$$

This reduces both storage and inference cost. It is the reason graphical models scale at all.

Why independence matters. Without independence assumptions, the joint distribution over many variables grows exponentially. Conditional independence is the mathematical device that turns an impossible joint table into a manageable model.

6.3 Bayesian networks

A Bayesian network is a directed acyclic graph whose factorization is

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid \mathrm{pa}(x_i)),$$

where $\mathrm{pa}(x_i)$ denotes the parents of node x_i.

The graph encodes more than a drawing. It says exactly how the joint distribution factorizes and which variables become irrelevant once certain others are known. The *Markov blanket* of a node consists of its parents, children, and co-parents of its children; conditioned on that set, the node is independent of the rest of the graph.

6.4 Bayesian prediction and model selection

In Bayesian learning, model parameters are themselves uncertain. If D is the dataset and $\mathbf{w}$ are parameters, then

$$p(\mathbf{w} \mid D) \propto p(D \mid \mathbf{w})\, p(\mathbf{w}).$$

Predictions average over that posterior:

$$p(y^* \mid \mathbf{x}^*, D) = \int p(y^* \mid \mathbf{x}^*, \mathbf{w})\, p(\mathbf{w} \mid D)\, \mathrm{d}\mathbf{w}.$$

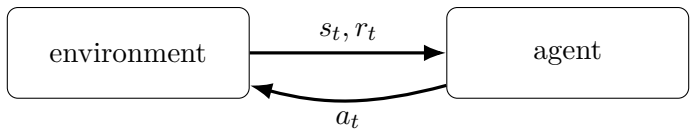

Figure 6: The reinforcement-learning loop: action changes future state distributions, so data are generated by the current policy itself.

This formula is one of the cleanest statements in the course. It says prediction should average over plausible models, not commit too early to a single one.

6.5 MAP estimation and regularization

If the full posterior integral is too costly, one may approximate it by a single parameter choice,

$$\mathbf{w}_{\mathrm{MAP}} = \arg\min_{\mathbf{w}} \left[-\log p(D \mid \mathbf{w}) - \log p(\mathbf{w}) \right].$$

This reveals the familiar bridge between Bayesian and frequentist viewpoints: a log-prior acts like a regularizer. For example, a Gaussian prior on $\mathbf{w}$ yields an ℓ_2 penalty.

6.6 What Bayesian methods add

The probabilistic block of the course contributes three durable ideas:

- uncertainty should be represented explicitly rather than hidden,
- graphical structure is a computational tool,
- regularization can be interpreted as prior information.

7 Reinforcement Learning

Reinforcement learning differs from supervised learning in one crucial way: the learner does not receive the correct target for each action immediately. It must act, observe consequences, and infer which decisions were responsible for later reward.

7.1 Markov decision processes

The standard formal model is a Markov decision process (MDP) with states s_t, actions a_t, rewards r_t, transition law $p(s_{t+1} \mid s_t, a_t)$, and policy $\pi(a \mid s)$.

The return from time t is

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \qquad 0 \le \gamma < 1.$$

Discounting makes distant reward less important and ensures convergence of the infinite-horizon sum under mild conditions.

7.2 Value functions

For policy π, the state-value and action-value functions are

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \qquad Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].$$

These satisfy Bellman equations such as

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V^\pi(s')\big].$$

Bellman recursion

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V^\pi(s')\big]$$

The value of a state is the expected immediate reward plus the discounted value of the next state. Reinforcement learning works because long-horizon evaluation can be written recursively.

7.3 Temporal-difference learning

Temporal-difference (TD) learning replaces the unknown future return by a one-step bootstrap target:

$$V(s_t) \leftarrow V(s_t) + \alpha\big[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)\big].$$

The bracketed term is the TD error. It measures how surprising the next transition was relative to the current value estimate.

7.4 Control: SARSA and Q-learning

For action values, two central updates are:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\big]$$

for SARSA, and

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big]$$

for Q-learning.

SARSA is on-policy: it learns the value of the behavior actually being followed. Q-learning is off-policy: it learns toward a greedy target even while behavior remains exploratory.

7.5 Exploration and exploitation

An agent that only exploits current knowledge may never discover better actions. An agent that only explores never consolidates what it has learned. Methods such as ε-greedy policies make this trade-off explicit:

- with probability $1 - \varepsilon$, choose the current best action,

- with probability ε, explore.

Why reinforcement learning is hard. The data distribution changes as the policy changes. In supervised learning, the dataset is usually fixed before optimization begins. In reinforcement learning, the learner partly creates its own future training data.

7.6 Course synthesis

The course closes on a useful unification:

- supervised learning studies prediction from examples,
- statistical learning theory studies when such prediction generalizes,
- Bayesian methods attach uncertainty to the predictions,
- reinforcement learning adds action and delayed credit assignment.

The common thread is always the same: choose a representation, define a principled objective, and use mathematics to separate what is merely fit from what is genuinely learned.

Mathematical Synthesis

Machine intelligence is built from a small number of recurring mathematical objects: data distributions, hypothesis classes, losses, optimization procedures, and uncertainty estimates. The course becomes easier to read when these are kept separate. Optimization asks whether we found a good parameter. Generalization asks whether that parameter will remain good away from the finite training sample.

Empirical Risk Is Only A Proxy

For a hypothesis h_θ and loss ℓ, training minimizes

$$\widehat{R}_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(h_\theta(x_i), y_i).$$

The real object of interest is the population risk

$$R(\theta) = \mathbb{E}_{(X,Y)\sim P}\left[\ell(h_\theta(X), Y)\right].$$

The empirical risk is computable; the population risk is what we actually care about. The gap between them is not a technical nuisance but the central difficulty of learning from data.

Learning problem

$$\theta^* \in \arg\min_\theta R(\theta), \qquad \hat{\theta}_n \in \arg\min_\theta \widehat{R}_n(\theta).$$

Learning succeeds when $\widehat{R}_n$ is informative enough that $\hat{\theta}_n$ also has small R. This requires data, inductive bias, and an optimizer that finds a useful region of parameter space.

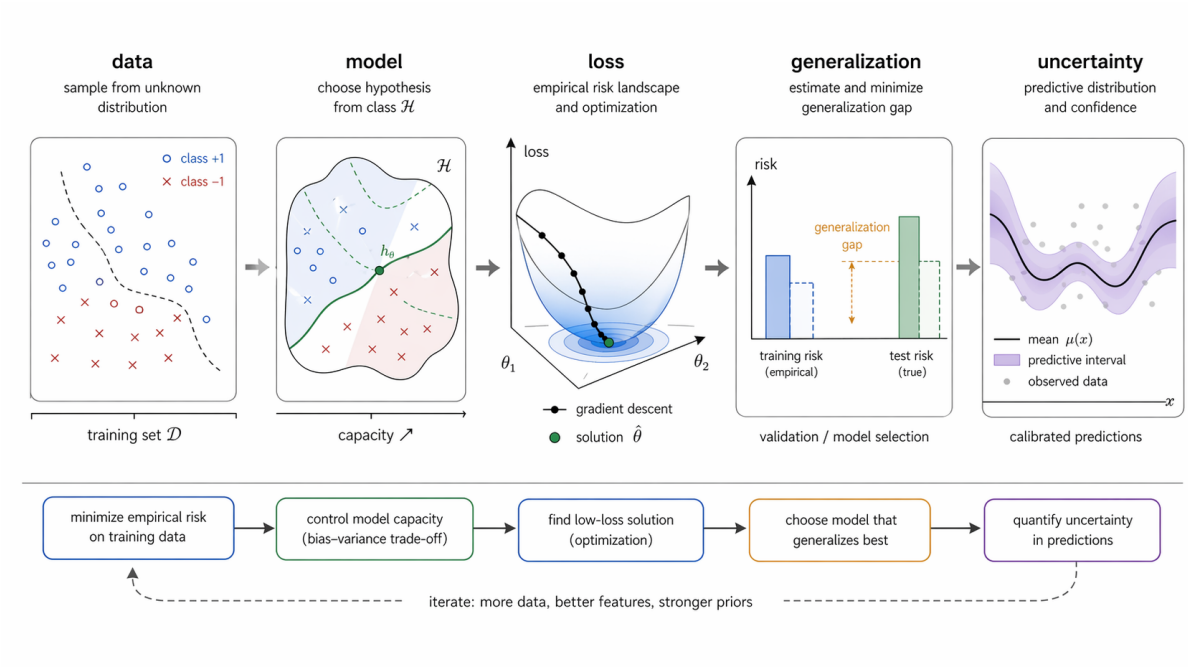

Figure 7: The main loop of supervised learning. Data define an empirical problem, a model class restricts the functions we can choose, optimization searches the empirical-risk landscape, validation estimates generalization, and uncertainty describes what predictions should be trusted.

Three Errors To Keep Apart

A crisp way to diagnose a model is to separate the failure modes:

$$R(\hat{\theta}) = \underbrace{R(\theta^*_{\mathcal{H}})}_{\text{approximation}} + \underbrace{R(\hat{\theta}_n) - R(\theta^*_{\mathcal{H}})}_{\text{estimation}} + \underbrace{R(\tilde{\theta}) - R(\hat{\theta}_n)}_{\text{optimization, if any}} .$$

The exact decomposition depends on notation, but the distinction is stable. Approximation error comes from the hypothesis class being too small or badly matched. Estimation error comes from finite data. Optimization error comes from failing to find a good empirical solution.

Deep learning often reduces approximation error by expanding the function class, then relies on architecture, data augmentation, regularization, and optimization bias to control the estimation problem.

Geometry Of Classification

For a linear classifier,

$$h(x) = \text{sign}(w^\top x + b),$$

the decision boundary is the hyperplane $w^\top x + b = 0$. The vector w is normal to the boundary, so changing w rotates the separator and changing b translates it. Margin methods turn this geometry into an objective: choose a boundary that classifies the data while leaving a large buffer around it.

This geometric picture survives inside nonlinear models. A neural network first transforms x into a representation $\phi_\theta(x)$, then makes a decision in that new coordinate system. The hard part is learning a representation in which the final decision boundary is simple.

Uncertainty Is Part Of The Output

A point prediction says what the model thinks. A distribution says how strongly it thinks it. Bayesian inference makes this explicit:

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}, \qquad p(y_* \mid x_*, D) = \int p(y_* \mid x_*, \theta)\, p(\theta \mid D)\, d\theta.$$

Even when exact Bayesian inference is not used, this formula is the right mental model. Good machine-intelligence systems should distinguish uncertainty caused by noisy observations from uncertainty caused by missing data. Calibration, validation curves, and posterior or ensemble predictions are all attempts to keep that distinction visible.

References

[Bis06] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[HTF09] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. 2nd ed. Springer, 2009.

[Vap98] Vladimir N. Vapnik. *Statistical Learning Theory*. Wiley, 1998.